

COMPLETE GUIDE · V2.0

# Kiro AI Chat

Step-by-step beginner guide to building a serverless AI chatbot on AWS

DEVELOPED BY

**Krishna Yada**

REGION

**ap-south-1**

VERSION

**2.0 — May 2026**

 Amazon Cognito

 AWS Lambda

 Amazon Bedrock

 API Gateway

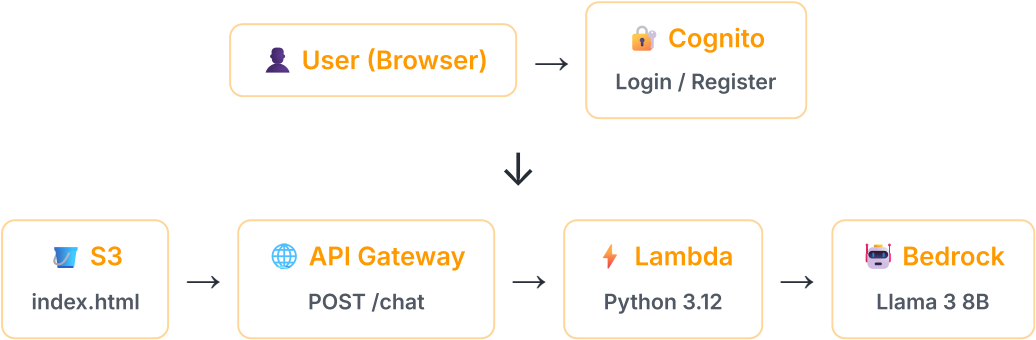
 Amazon S3

## **Table of Contents**

1. **1.** Architecture Overview
2. **2.** Prerequisites
3. **3.** Step 1 — Create Cognito User Pool
4. **4.** Step 2 — Create IAM Role for Lambda
5. **5.** Step 3 — Deploy Lambda Function (Meta Llama 3)
6. **6.** Step 4 — Create API Gateway (No Auth)
7. **7.** Step 5 — Build & Configure the Webpage
8. **8.** Step 6 — Host on S3 (Public URL)
9. **9.** Troubleshooting
10. **10.** Live Resources Summary

# 1 Architecture Overview

Five AWS services work together. Here's the full data flow:



| Service        | Role                                          |
|----------------|-----------------------------------------------|
| Amazon Cognito | User registration, login, email verification  |
| AWS Lambda     | Python function that calls Bedrock AI         |
| Amazon Bedrock | Runs Meta Llama 3 8B to generate AI responses |
| API Gateway    | Public HTTPS endpoint that triggers Lambda    |
| Amazon S3      | Hosts the HTML chat UI as a public website    |

## **i** Key design decision (v2.0)

API Gateway has NO Cognito authorizer — authentication is handled by Cognito on the frontend only. This avoids token expiry issues on HTTP-hosted pages.

## 2 Prerequisites

---

### A AWS Account

Sign up free at [aws.amazon.com](https://aws.amazon.com)

### B AWS CLI installed & configured

Download from [aws.amazon.com/cli/](https://aws.amazon.com/cli/), then run:

#### Terminal

Run once

```
aws configure
# Enter: Access Key, Secret Key, region: ap-south-1, output: json
```

### C Python 3.x

Download from [python.org](https://python.org)

### D A text editor

VS Code recommended

## 3 Step 1 — Create Cognito User Pool

---

### 1.1 Create User Pool

#### Terminal

 Copy & Run

```
aws cognito-idp create-user-pool \  
  --pool-name kiro-chat-pool \  
  --username-attributes email \  
  --auto-verified-attributes email \  
  --policies "PasswordPolicy={MinimumLength=8,RequireUppercase=true,RequireLowercase=true,RequireNumbers=true,RequireSymbols=true}" \  
  --region ap-south-1
```

#### ⚠ Save the "Id" from output

Looks like: ap-south-1\_XXXXXXX — this is your UserPoolId

### 1.2 Create App Client

#### Terminal

 Replace YOUR\_USER\_POOL\_ID

```
aws cognito-idp create-user-pool-client \  
  --user-pool-id YOUR_USER_POOL_ID \  
  --client-name kiro-chat-client \  
  --no-generate-secret \  
  --explicit-auth-flows ALLOW_USER_PASSWORD_AUTH ALLOW_REFRESH_TOKEN_AUTH ALLOW_USER_SRP_AUTH \  
  --prevent-user-existence-errors ENABLED \  
  --region ap-south-1
```

#### ⚠ Save the "ClientId" from output

Long alphanumeric string — needed in the webpage config

### 1.3 Manually confirm a user (if "User is not confirmed" error)

#### Terminal

 Run when needed

```
aws cognito-idp admin-confirm-sign-up \  
  --user-pool-id YOUR_USER_POOL_ID \  
  --username USERNAME
```

```
--username user@example.com \  
--region ap-south-1
```

✓ **Use this for any user who registered but can't sign in**

No output = success.

## 4 Step 2 — Create IAM Role for Lambda

---

### 4.1 Create trust-policy.json

Create a file named **trust-policy.json** with this content:

**trust-policy.json**

 Create this file

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "lambda.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

### 4.2 Create role and attach policies

**Terminal**

 Run all 3 commands

```
aws iam create-role \
  --role-name kiro-lambda-bedrock-role \
  --assume-role-policy-document file://trust-policy.json

aws iam attach-role-policy \
  --role-name kiro-lambda-bedrock-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

aws iam attach-role-policy \
  --role-name kiro-lambda-bedrock-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonBedrockFullAccess
```

 **No output = success for the attach commands**

## 5 Step 3 — Deploy Lambda Function

Uses **Meta Llama 3 8B** — the only model confirmed working in ap-south-1 without extra approvals.

### ⚠ Do NOT use these models — they are broken in ap-south-1:

- `amazon.titan-text-express-v1` — End of life
- `amazon.nova-micro-v1:0` — Different API format
- `anthropic.claude-*` — Requires Anthropic use-case form approval

### 5.1 Create `lambda_function.py`

`lambda_function.py`

Create this file

```
import json
import boto3

bedrock = boto3.client("bedrock-runtime", region_name="ap-south-1")

def lambda_handler(event, context):
    headers = {
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Headers": "Content-Type",
        "Access-Control-Allow-Methods": "POST,OPTIONS"
    }
    if event.get("httpMethod") == "OPTIONS":
        return {"statusCode": 200, "headers": headers, "body": ""}
    try:
        body = json.loads(event.get("body") or "{}")
        msg = body.get("message", "").strip()
        response = bedrock.invoke_model(
            modelId="meta.llama3-8b-instruct-v1:0",
            contentType="application/json",
            accept="application/json",
            body=json.dumps({
                "prompt": f"<begin_of_text> <start_header_id>user<end_header_id>\n{msg}<eot_id> <start_header_id>assistant<end_header_id>\n",
                "max_gen_len": 1024, "temperature": 0.7
            })
        )
        result = json.loads(response["body"].read())
```

```

        return {"statusCode": 200, "headers": headers,
                "body": json.dumps({"reply": result["generation"].strip()})}
    except Exception as e:
        return {"statusCode": 500, "headers": headers,
                "body": json.dumps({"error": str(e)})}

```

## 5.2 Zip and deploy

### PowerShell (Windows)

 Copy & Run

```
Compress-Archive -Force -Path lambda_function.py -DestinationPath lambda.zip
```

### Terminal – replace YOUR\_ACCOUNT\_ID

 Copy & Run

```

aws lambda create-function \
  --function-name kiro-bedrock-chat \
  --runtime python3.12 \
  --role arn:aws:iam::YOUR_ACCOUNT_ID:role/kiro-lambda-bedrock-role \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://lambda.zip \
  --timeout 30 \
  --region ap-south-1

```

#### Find your Account ID

```
aws sts get-caller-identity --query Account --output text
```

## 5.3 Update Lambda code (if you make changes)

### Terminal

 Run after editing lambda\_function.py

```

Compress-Archive -Force -Path lambda_function.py -DestinationPath lambda.zip
aws lambda update-function-code \
  --function-name kiro-bedrock-chat \
  --zip-file fileb://lambda.zip \
  --region ap-south-1

```

## 6 Step 4 — Create API Gateway

### v2.0 Change — No Cognito Authorizer

The API Gateway uses **NONE** authorization. Cognito is used only for the login UI. This prevents 401 errors caused by token expiry on HTTP-hosted pages.

#### Terminal

 Run all commands in order – replace placeholders

*# 1. Create API – save the "id" as API\_ID*

```
aws apigateway create-rest-api --name kiro-chat-api \  
  --endpoint-configuration types=REGIONAL --region ap-south-1
```

*# 2. Get root resource – save "id" as ROOT\_ID*

```
aws apigateway get-resources --rest-api-id API_ID --region ap-south-1
```

*# 3. Create /chat resource – save "id" as RESOURCE\_ID*

```
aws apigateway create-resource --rest-api-id API_ID \  
  --parent-id ROOT_ID --path-part chat --region ap-south-1
```

*# 4. Add POST method with NO auth*

```
aws apigateway put-method --rest-api-id API_ID \  
  --resource-id RESOURCE_ID --http-method POST \  
  --authorization-type NONE --region ap-south-1
```

*# 5. Connect Lambda*

```
aws apigateway put-integration --rest-api-id API_ID \  
  --resource-id RESOURCE_ID --http-method POST \  
  --type AWS_PROXY --integration-http-method POST \  
  --uri arn:aws:apigateway:ap-south-1:lambda:path/2015-03-31/functions/arn:aws:lambda:ap-south-1:YOUR_ACCOUNT_ID:function:kiro-bedrock-chat/invocations \  
  --region ap-south-1
```

*# 6. Allow API Gateway to invoke Lambda*

```
aws lambda add-permission --function-name kiro-bedrock-chat \  
  --statement-id apigateway-invoke \  
  --action lambda:InvokeFunction \  
  --principal apigateway.amazonaws.com \  
  --source-arn "arn:aws:execute-api:ap-south-1:YOUR_ACCOUNT_ID:API_ID/*//*/chat" \  
  --region ap-south-1
```

*# 7. Deploy*

```
aws apigateway create-deployment --rest-api-id API_ID \  
  --stage-name prod --region ap-south-1
```

✅ **Your API endpoint:**

[https://API\\_ID.execute-api.ap-south-1.amazonaws.com/prod/chat](https://API_ID.execute-api.ap-south-1.amazonaws.com/prod/chat)

## 7 Step 5 — Configure the Webpage

---

Open **index.html** and find these 3 lines near the top of the script block. Replace with your real values:

**index.html**

 Edit these 3 lines

```
const API      = "https://API_ID.execute-api.ap-south-1.amazonaws.com/prod/c  
hat";  
const POOL_ID  = "ap-south-1_XXXXXXXXXX";  
const CLIENT_ID = "XXXXXXXXXXXXXXXXXXXXXXXXXXXX";
```

### Key features of the webpage (v2.0)

- Cognito login/register with email verification
- Password strength meter with live checklist
- Show/hide password toggle
- Quick-prompt chips that send immediately on click
- Chat sends with Enter key, Shift+Enter for new line
- Animated typing indicator while AI responds
- No token dependency — works on HTTP and HTTPS
- "Developed by Krishna Yada" credit in footer

#### **Test locally first**

Double-click index.html to open in browser. Sign in and test chat before uploading to S3.

## 8 Step 6 — Host on S3 (Public URL)

---

### Terminal

 Replace YOUR-BUCKET-NAME throughout

#### *# Create bucket*

```
aws s3api create-bucket --bucket YOUR-BUCKET-NAME --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1
```

#### *# Allow public access*

```
aws s3api put-public-access-block --bucket YOUR-BUCKET-NAME \
  --public-access-block-configuration \
    BlockPublicAcls=false,BlockPublicPolicy=false,IgnorePublicAcls=false,RestrictPublicBuckets=false
```

```
aws s3api put-bucket-policy --bucket YOUR-BUCKET-NAME \
  --policy '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":"*", "Action":"s3:GetObject","Resource":"arn:aws:s3:::YOUR-BUCKET-NAME/*"}]}'
```

#### *# Enable static website*

```
aws s3api put-bucket-website --bucket YOUR-BUCKET-NAME \
  --website-configuration '{"IndexDocument":{"Suffix":"index.html"},"ErrorDocument":{"Key":"index.html"}}'
```

#### *# Upload*

```
aws s3 cp index.html s3://YOUR-BUCKET-NAME/index.html \
  --content-type text/html --cache-control "no-cache"
```

#### **Public URL:**

<http://YOUR-BUCKET-NAME.s3-website.ap-south-1.amazonaws.com>

Share this with anyone — no AWS account needed to use the app!

#### **To update the page after edits:**

Re-run the `aws s3 cp` command, then press **Ctrl+Shift+R** in the browser.

## 9 Troubleshooting

---

| Error / Symptom                       | Cause                                    | Fix                                               |
|---------------------------------------|------------------------------------------|---------------------------------------------------|
| User is not confirmed                 | Email verification not done              | Run admin-confirm-sign-up (Step 1.3)              |
| Chat sends but no response appears    | Old cached page in browser               | Hard refresh: Ctrl+Shift+R                        |
| 401 Unauthorized from API             | Cognito authorizer on API Gateway        | Set authorization to NONE and redeploy (Step 4)   |
| 403 Forbidden from API                | Lambda resource policy missing           | Re-run add-permission command (Step 4, command 6) |
| ResourceNotFoundException — model EOL | Titan model deprecated                   | Use meta.llama3-8b-instruct-v1:0 only             |
| Anthropic use case form error         | Claude needs approval                    | Use Meta Llama 3 instead                          |
| Header shows UUID not email           | Using getUsername() which returns sub    | Use session.getIdToken().payload.email            |
| Chips don't generate output           | fillInput() only filled box, didn't send | Use ask(text) which calls send() directly         |

# 10 Live Resources Summary

| Resource           | Name                                                                       | Value                                                 |
|--------------------|----------------------------------------------------------------------------|-------------------------------------------------------|
| Cognito User Pool  | kiro-chat-pool                                                             | ap-south-1_IFldpENX0                                  |
| Cognito App Client | kiro-chat-client                                                           | 7gm2k1ckmq1rv1erp5mkilcok3                            |
| IAM Role           | kiro-lambda-bedrock-role                                                   | AWSLambdaBasicExecutionRole + AmazonBedrockFullAccess |
| Lambda Function    | kiro-bedrock-chat                                                          | Python 3.12 · 30s timeout · Meta Llama 3              |
| API Gateway        | kiro-chat-api                                                              | uadyf15gch · Auth: NONE                               |
| S3 Bucket          | kiro-chat-webapp-krishna                                                   | Static website hosting enabled                        |
| Public URL         | <b>http://kiro-chat-webapp-krishna.s3-website.ap-south-1.amazonaws.com</b> |                                                       |



## Congratulations!

You've built a fully working serverless AI chat app on AWS. Share the public URL with anyone!